

Data Management Mini Toolkit (DMMT) Report

Course: Data Structures (ETCCDS202)

Unit 4: Non-Linear & Advanced Data Structures

Student Name: Tushar saini

Roll No: 2501730053

1. Binary Search Tree (BST) Operations

The BST module implements an ordered structure supporting efficient search and sorted traversal.

- **Inorder Traversal and Sorting:** Inorder traversal visits nodes in the order: Left Subtree → Root → Right Subtree. Because a BST is structured such that all keys in the left subtree are smaller than the root and all keys in the right subtree are larger, this traversal naturally prints keys in non-decreasing sorted order.
 - **Time Complexity Discussion:**
 - **Average Case:** $O(\log n)$ for search, insert, and delete operations, as the height of a balanced tree is logarithmic relative to the number of nodes.
 - **Worst Case:** $O(n)$ if the tree becomes "skewed" (resembling a linked list), which happens when elements are inserted in already sorted or reverse-sorted order.
-

2. Graph Representation and Traversal

The toolkit uses an **Adjacency List** to represent a directed, weighted graph with 6 nodes and 9 edges.

- **BFS vs. DFS: Breadth-First Search (BFS):** Explores the neighbor nodes first, before moving to the next level neighbors. It uses a queue-based approach to visit nodes in level-order.
 - **Depth-First Search (DFS):** Explores as far as possible along each branch before backtracking. It uses recursion or a stack-based approach.
- **Real-World Use Cases:**
 - **BFS:** Used for finding the shortest path in an unweighted graph (e.g., finding the fewest "hops" between two people in a social network).

- **DFS:** Used for cycle detection in a network or solving puzzles like mazes where you explore one path completely before trying another.
-

3. Hash Table with Separate Chaining

The Hash Table module provides fast key-value storage using a modulo-based hash function ($\text{index} = \text{key} \pmod{5}$).

- **Collisions:** A collision occurs when two different keys result in the same index after being processed by the hash function. For example, with a table size of 5, keys 10, 15, and 20 all collide at index 0.
 - **Separate Chaining:** This method handles collisions by maintaining a list (or "chain") of elements at each index of the table. Instead of overwriting data, new items with the same hash index are simply appended to the existing list.
 - **Time Complexity:** **Average Case:** $O(1)$ for insert, get, and delete operations.
 - **Worst Case:** $O(n)$ if many keys collide into the same bucket, effectively turning the hash table into a long linked list.
-

4. References

- Unit-4 Course Material: Non-Linear & Advanced Data Structures.
- Python Documentation: `collections.deque` for efficient queue operations.